

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Florea David, FAF-242

Report

Laboratory work nr. 1

The OS as a Resource Manager

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Setup

Track A (Linux). Ubuntu 26.04.1 LTS running in VirtualBox, user `david`.

```
mkdir -p ~/os-lab1 && cd ~/os-lab1
whoami
uname -a
uptime
```

```
david
Linux david-VirtualBox 7.0.0-38-generic #38-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep  4
    09:10:14 UTC 2026 x86_64 GNU/Linux
 14:13:02 up  1:28,  1 user,  load average: 1.57, 1.67, 1.47
```

Part 1 – Files and Directories

```
ls -la /
```

```
drwxr-xr-x  20 root root       4096 Oct  5 12:15 .
lrwxrwxrwx   1 root root          7 Apr 20 11:46 bin -> usr/bin
drwxr-xr-x   3 root root       4096 Oct  5 12:25 boot
drwxr-xr-x  19 root root       4160 Oct  5 14:05 dev
drwxr-xr-x 141 root root      12288 Oct  5 12:57 etc
drwxr-xr-x   3 root root       4096 Oct  5 12:18 home
lrwxrwxrwx   1 root root          7 Apr 20 11:46 lib -> usr/lib
dr-xr-xr-x 251 root root          0 Oct  5 12:41 proc
lrwxrwxrwx   1 root root          8 Apr 20 11:46 sbin -> usr/sbin
-rw-------   1 root root 3530555392 Oct  5 12:15 swap.img
dr-xr-xr-x  13 root root          0 Oct  5 12:41 sys
drwxrwxrwt  17 root root        400 Oct  5 14:12 tmp
drwxr-xr-x  13 root root       4096 Oct  5 12:57 usr
drwxr-xr-x  14 root root       4096 Oct  5 12:29 var
...
```

```
cd /etc && ls | head -6
```

```
ModemManager
NetworkManager
PackageKit
UPower
X11
adduser.conf
```

```
cd ~/os-lab1 && mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
ls -l
```

```
total 8
-rw-rw-r-- 1 david david 24 Oct  5 14:13 note.txt
-rw-rw-r-- 1 david david 24 Oct  5 14:13 renamed.txt
```

```
total 4
-rw-rw-r-- 1 david david 24 Oct  5 14:13 note.txt
```

```
chmod 600 note.txt && ls -l note.txt
chmod 644 note.txt && ls -l note.txt
id
```

```
-rw------- 1 david david 24 Oct  5 14:13 note.txt
-rw-r--r-- 1 david david 24 Oct  5 14:13 note.txt
uid=1000(david) gid=1000(david) groups=1000(david),4(adm),24(cdrom),27(sudo),...
```

Observations

(1) Owner and group. The files I created (`note.txt`, `renamed.txt`) are owned by user **david** and group **david**. On Ubuntu every user gets a private group with the same name (`id` shows `uid=1000(david) gid=1000(david)`).

(2) The ten characters after `chmod 600`. `ls -l note.txt` printed `-rw----`:

`-`	file type: regular file (`d` = directory, `l` = symlink, `c`/`b` = device)
`rw-`	owner (david) may read and write but not execute
`--`	group (david) gets no access
`--`	others get no access

So after `chmod 600` only I can read or modify the file. After `chmod 644` it became `-rw-r-r-`: everyone can read it but only I can write it.

(3) Two directories under `/`.

- `/etc` holds system-wide configuration files (e.g. `adduser.conf`, `NetworkManager/`, `X11/`).
- `/home` holds the users' personal home directories (here `/home/david`).
- (Bonus) `/proc` is not on disk at all. It's a virtual filesystem the kernel generates, with one directory per running process, which is why its size shows as `0`.

Interesting output: on this Ubuntu, `/bin`, `/lib` and `/sbin` are just symlinks into `/usr` (the "merged /usr" layout), and swap is a 3.3 GB file (`/swap.img`) rather than a partition.

Part 2 – Processes

```
ps aux | head -4
ps aux | wc -l
top -b -n 1 | head -12
```

```
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.3  25920 13600 ?        Ss   12:44   0:03 /usr/lib/systemd/
    systemd --switched-root --system --deserialize=52 splash
root           2  0.0  0.0      0     0 ?        S    12:44   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    12:44   0:00 [
    pool_workqueue_release]
```

```
241
top - 14:13:32 up  1:28,  1 user,  load average: 1.30, 1.60, 1.45
Tasks: 240 total,   1 running, 238 sleeping,   0 stopped,   1 zombie
%Cpu(s):  5.9 us,  5.9 sy,  0.0 ni, 82.4 id,  0.0 wa,  0.0 hi,  5.9 si,  0.0 st
MiB Mem :   3398.8 total,    185.6 free,   1872.1 used,   1498.8 buff/cache
MiB Swap:   3367.0 total,   2388.5 free,    978.5 used.   1526.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   3602 david     20   0 1865644 101020  72224 S  18.2   2.9   0:28.22 ptyxis
   2594 david     20   0 4053412 247252 103592 S   9.1   7.1   4:02.74 gnome-s+
  11337 david     20   0   13380   5848   3792 R   9.1   0.2   0:00.01 top
      1 root      20   0   25920  13600   9576 S   0.0   0.4   0:03.51 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.02 kthreadd
```

```
sleep 300 &
jobs -l
ps -ef | grep sleep
kill 11339
jobs -l
```

```
[1] 11339
[1]+ 11339 Running                 sleep 300 &
david      11339   11331  0 14:13 ?        00:00:00 sleep 300
david      11341   11331  0 14:13 ?        00:00:00 grep sleep
[1]+ 11339 Terminated              sleep 300
```

```
sleep 300 & PID=$!
ls /proc/$PID/ | head -5
cat /proc/$PID/status | head -6
kill $PID
ps -p 1 -o pid,user,comm,args
ls -l /sbin/init
```

```
arch_status
attr
autogroup
auxv
cgroup
Name:   sleep
Umask:  0002
State:  S (sleeping)
Tgid:   11344
Ngid:   0
Pid:    11344
    PID USER     COMMAND         COMMAND
      1 root     systemd         /usr/lib/systemd/systemd --switched-root --system --
    deserialize=52 splash
lrwxrwxrwx 1 root root 22 Jul 27 23:44 /sbin/init -> ../lib/systemd/systemd
```

Observations

(1) PID 1. PID 1 is **systemd** (run by root; `/sbin/init` is a symlink to it). It's the first user-space process the kernel starts at boot (the *init* process). It starts and supervises all the system services, and it adopts orphaned processes. PID 2 is `kthreadd`, the parent of all kernel threads.

(2) Number of processes. `ps aux | wc -l` returned **241**. Subtracting the header line leaves **240 processes** on an "idle" VM, which matches `top`: `Tasks: 240 total, 1 running, 238 sleeping, 1 zombie`. Most of them are kernel threads (`[kworker/...]`, `[rcu_...]`) and GNOME desktop processes.

(3) State of the sleeping process. `/proc/<pid>/status` showed `State: S (sleeping)`. `sleep 300` isn't using the CPU. It's blocked in the kernel waiting for a timer to expire, so the scheduler doesn't run it until then.

Interesting output: `kill` doesn't kill anything by itself. It asks the kernel to deliver signal 15 (SIGTERM) to the process, and the kernel then terminates it, which `jobs -l` reports as `Terminated`.

Part 3 – Memory

```
free -h
cat /proc/meminfo | head -6
swapon --show
```

```
               total        used        free      shared  buff/cache   available
Mem:           3.3Gi       1.8Gi       192Mi        39Mi       1.5Gi       1.5Gi
Swap:          3.3Gi       978Mi       2.3Gi
MemTotal:        3480328 kB
MemFree:          197556 kB
MemAvailable:    1576952 kB
Buffers:            9060 kB
Cached:          1457600 kB
SwapCached:       104292 kB
NAME      TYPE SIZE   USED PRIO
/swap.img file 3.3G 978.4M   -1
```

```
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
grep -E "VmSize|RssAnon|RssFile|RssShmem" /proc/$PID/status
kill $PID
```

```
VmRSS:      7740 kB
VmSize:    16116 kB
RssAnon:             1224 kB
RssFile:             6516 kB
RssShmem:               0 kB
```

Observations

(1) Total and free RAM. `free -h`: **3.3 GiB total** (MemTotal 3 480 328 kB), 1.8 GiB used, **192 MiB "free"** but **1.5 GiB "available"**. "Free" is low because Linux uses otherwise idle RAM as page cache (`buff/cache` 1.5 GiB). That cache is released when programs need memory, so "available" is the real headroom.

(2) Swap. Swap is disk space the kernel uses as an overflow for RAM: rarely used memory pages are moved out to disk to make room in physical memory. This VM has a **3.3 GiB swap file** (`/swap.img`, not a partition), of which about 978 MiB was in use. The VM is fairly memory-constrained with a desktop running.

(3) VmRSS of `sleep`. `VmRSS: 7740 kB` (about 7.6 MB), while the virtual size was `VmSize: 16116 kB`. That surprised me for a program that does nothing. The breakdown explains it: `RssAnon: 1224 kB` is its own private memory and `RssFile: 6516 kB` is mapped files, mostly the C library and the `sleep` binary. Those file pages are **shared** with every other process using libc, so `sleep` only costs about 1.2 MB of extra RAM.

Part 4 – Devices and Storage

```
df -h
lsblk
findmnt /
```

```
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           680M  2.4M  678M   1% /run
/dev/sda2        20G   12G  7.5G  60% /
tmpfs           1.7G  4.0K  1.7G   1% /dev/shm
tmpfs           1.7G  5.0M  1.7G   1% /tmp
tmpfs           340M  5.4M  335M   2% /run/user/1000
/dev/sr0         51M   51M     0 100% /run/media/david/VBox_GAs_7.2.20
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda      8:0    0    20G  0 disk
|-sda1   8:1    0     1M  0 part
`-sda2   8:2    0    20G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/david/VBox_GAs_7.2.20
TARGET SOURCE    FSTYPE OPTIONS
/      /dev/sda2 ext4   rw,relatime
```

(Output shortened: the two `none` credential mounts from `df` and the 13 `loop` devices from `lsblk`, which belong to snap packages, are omitted.)

```
ls -l /dev | head -6
ls -l /dev/null /dev/sda /dev/tty /dev/urandom
```

```
total 0
crw-r--r--  1 root  root     10, 235 Oct  5 12:41 autofs
drwxr-xr-x  2 root  root        400 Oct  5 13:49 block
drwxr-xr-x  2 root  root         80 Oct  5 12:41 bsg
crw-------  1 root  root     10, 234 Oct  5 12:41 btrfs-control
drwxr-xr-x  3 root  root         60 Oct  5 13:40 bus
crw-rw-rw- 1 root root 1, 3 Oct  5 12:41 /dev/null
brw-rw---- 1 root disk 8, 0 Oct  5 12:41 /dev/sda
crw-rw-rw- 1 root tty  5, 0 Oct  5 12:57 /dev/tty
crw-rw-rw- 1 root root 1, 9 Oct  5 12:41 /dev/urandom
```

Observations

(1) Root filesystem device. `/` is mounted from `/dev/sda2`, an **ext4** partition of 20 GB (12 GB used, 60%). `lsblk` shows that `sda` is the VM's single 20 GB virtual disk, with `sda1` a tiny 1 MB BIOS-boot partition and `sda2` the root.

(2) A `/dev` entry.

- `/dev/sda` (`brw-rw-- root disk 8, 0`) is the whole virtual hard disk: `b` = block device, major number 8 = SCSI/SATA disk driver.

- `/dev/null` (`crw-rw-rw- 1, 3`) is a character device that discards anything written to it.
- `/dev/sr0` is the virtual CD drive, currently holding the VirtualBox Guest Additions ISO.

(3) "Everything is a file". Disks, terminals, the CD-ROM, even random numbers (`/dev/urandom`) and process information (`/proc`) are all exposed as entries in the same directory tree. Programs use them through the same `open/read/write/close` calls they use for ordinary files, and the kernel's drivers do the device-specific work behind that interface.

Interesting output: `df -h` shows that several "filesystems" don't live on any disk at all. They are RAM-backed `tmpfs` mounts (`/run`, `/tmp`, `/dev/shm`).

Optional – htop and strace

```
htop        # captured via a detached GNU screen session
```

```
    0[||||                       10.4%] Tasks: 148, 642 thr, 89 kthr; 1 runnin
    1[|||                         6.8%] Load average: 0.89 1.44 1.41
  Mem[||||||||||||||||||||1.69G/3.32G] Uptime: 01:29:53
  Swp[||||||||||            877M/3.29G]
```

```
strace -c ls
```

```
demo
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ------------------
 30.18    0.000497         497         1           execve
 26.29    0.000433          16        27           mmap
 18.52    0.000305         305         1           write
  5.28    0.000087           7        11         1 openat
  4.19    0.000069           7         9           fstat
  3.95    0.000065           6        10           close
  3.76    0.000062           4        14           read
  2.73    0.000045           9         5           brk
  2.00    0.000033           4         8           mprotect
  ...
  0.00    0.000000           0         7         7 ioctl
  0.00    0.000000           0         2           getdents64
------ ----------- ----------- --------- --------- ------------------
100.00    0.001647          11       138        12 total
```

`htop 3.4.1` shows the same data as `top`, but with per-CPU bars (2 vCPUs here), memory/swap bars, and a scrollable process list sorted by CPU.

`strace -c ls` made **138 system calls (12 of them failed)** just to print one word:

- `mmap` ×27, `openat` ×11, `read` ×14 and `mprotect` ×8 load the program and its shared libraries into memory;
- `getdents64` ×2 is the call that actually reads the directory entries;
- `write` ×1 prints `demo` to stdout;

- the 7 failed `ioctl` calls are `ls` checking whether stdout is a terminal (it wasn't: `ENOTTY`).

So almost all of the work is setup. The "real" job of `ls` comes down to `openat` + `getdents64` + `write`, and every one of those is a request to the kernel.

Conclusion

In this laboratory work I looked at Linux from the outside, as a manager of four kinds of resources. It manages **files**: `ls -l` and `chmod` showed it naming data in one directory tree and enforcing who may read or write each file. It manages **processes**: `ps aux` and `top` showed about 240 processes that it creates, schedules and signals (`kill`), all visible under `/proc`. It manages **memory**: `free -h` and `VmRSS` showed it splitting 3.3 GiB of RAM plus swap between processes and sharing library pages between them. It manages **devices**: `df -h`, `lsblk` and `ls -l /dev` showed it hiding the disk and other hardware behind file-like interfaces. Finally, `strace` showed that even a trivial command like `ls` gets everything it needs from the kernel through system calls.